

Part B - RNN

RNN-Based Movie Review Sentiment Classification

By: Ng Yu Hang | P2423500 | DAAA/FT/2B/23

Project information + Goals

Project Information:

- Build an RNN model, to predict whether a movie review is a **positive/neutral/negative** review based on the given dataset.

Goals:

- Make an **experiment + report-style** notebook that:
 - Is **reproducible** (consistent results with the use of set seeds (42))
 - **Modular** (pre-defined functions)
 - Is very **scalable** (Polarity checker – automatically corrects scores)
 - Clean and structured experimental design phases.
 - Has **clean and clear documentation** with **explainable** models
 - Follows **industry-standard practices** (informative model naming + centralized data augmentation pipelines + auto-polarity label check)

```
SEED = 42

np.random.seed(SEED)
random.seed(SEED)
tf.random.set_seed(SEED)
os.environ['PYTHONHASHSEED'] = str(SEED)
```

Our Dataset – Movie Reviews

Review	Score	Are there ways for you to generate more data? Splitting up sentences, would that help?	Language
I'm thrilled throughout the movie. Engaging storytelling!	0.15	NaN	English
Avengers: Endgame mengecewakan dan terlalu klise. Kurang memuaskan.	0.85	NaN	Malay
Complex but very interesting plot. Big surprise at the end!	0.08	NaN	English
Amazing	0.12	NaN	English
Complex	0.1	NaN	English
Overwhelming action	0.4	NaN	English

Our Dataset – Movie Reviews

Review	Score	Are there ways for you to generate more data? Splitting up sentences, would that help?	Language
I'm thrilled throughout the movie. Engaging storytelling!	0.15	NaN	English
Avengers: Endgame mengecewakan dan terlalu klise. Kurang memuaskan.	0.85	NaN	Malay
Complex but very interesting plot. Big surprise at the end!	0.08	NaN	English
Amazing	0.12	NaN	English
Complex	0.1	NaN	English
Overwhelming action	0.4	NaN	English

- Some scores doesn't seem to make sense.

But we do not have information that they are reversed (i.e. high scores = high negativity).

Assume that they are lots of input errors rather than reversed.

We will deal with this later with an automatic polarity tone checker

Useless Column

Project Approach

- **Using 3-class classification approach**
 - **Because**
 1. **Extremely inconsistent labelling** - causes greater effects in regression than classification as classification separates them into classes.
 2. **Small data** - regression needs large amounts of data since its predicting a precise value
- **Translate Malay into English, rather than developing a multi-language model**
 - **Small dataset already**
 - **Better to train a specialized, good model in one language, than an all in one model.**

Project Approach – 3 class approach

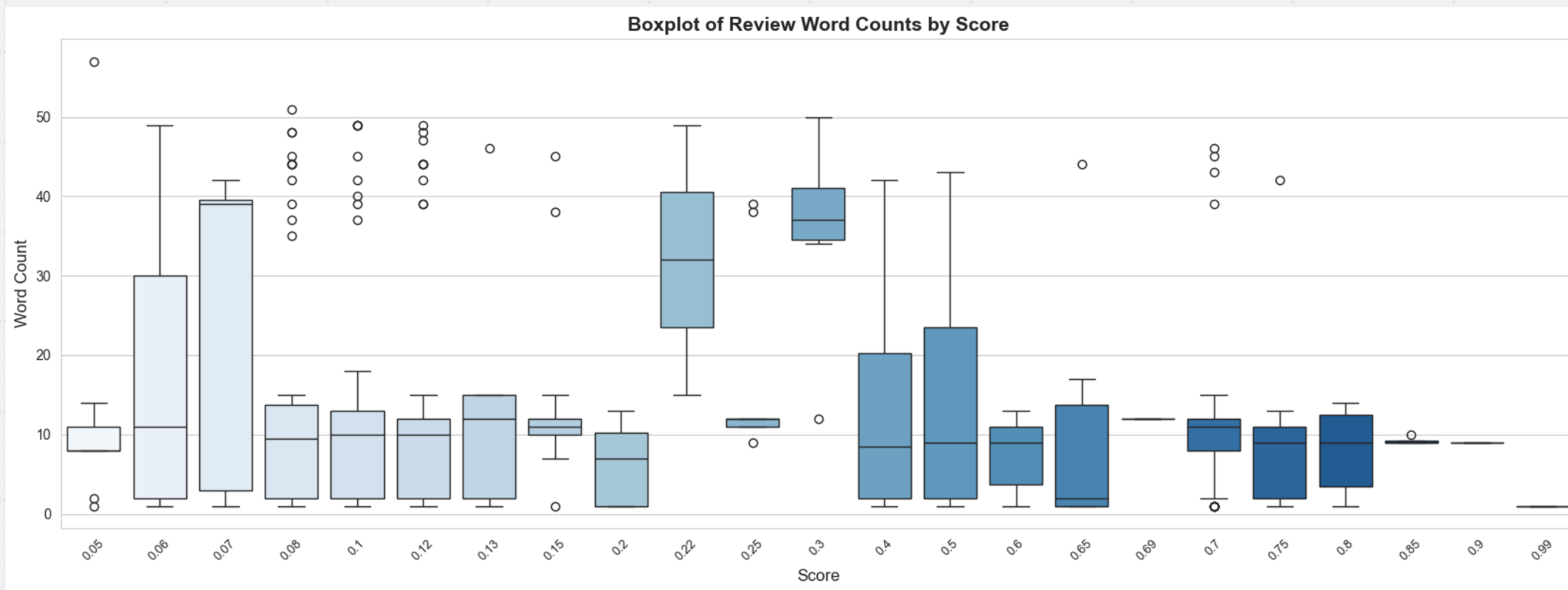
Positive: *Score* > 0.6

Neutral: 0.4 < *Score* < 0.6

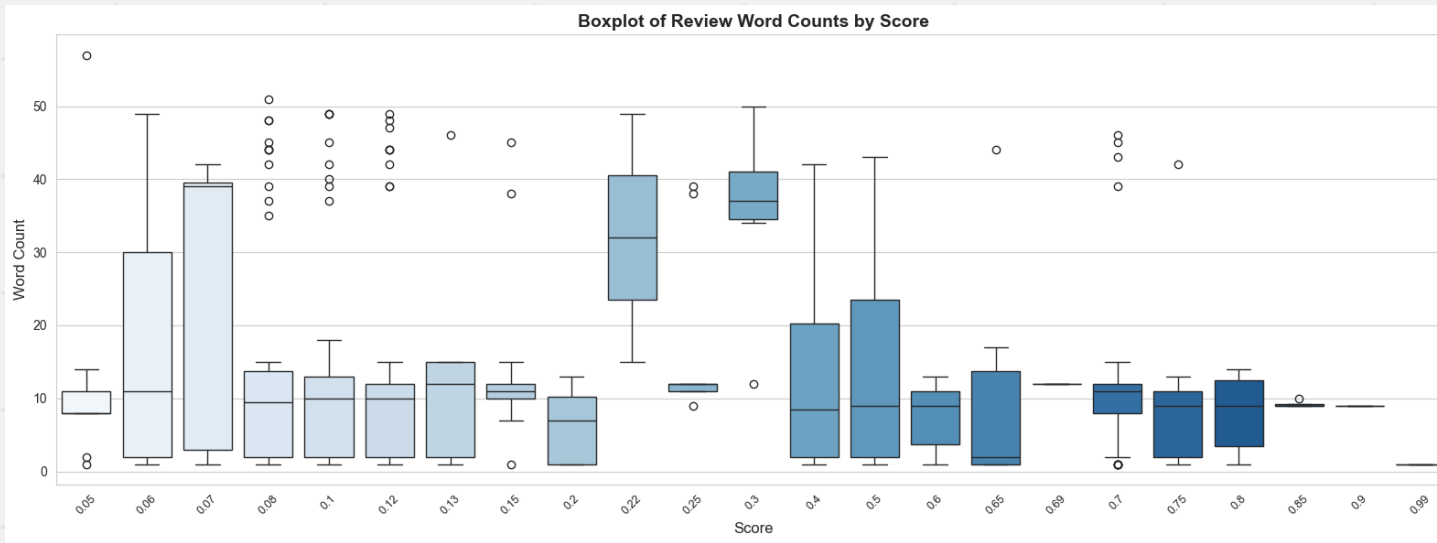
Negative: *Score* < 0.4

```
def bin_scores_scaled(score):  
    if score <= 0.4:  
        return 0 # Negative  
    elif score < 0.6:  
        return 1 # Neutral  
    else:  
        return 2 # Positive
```

Exploratory Data Analysis



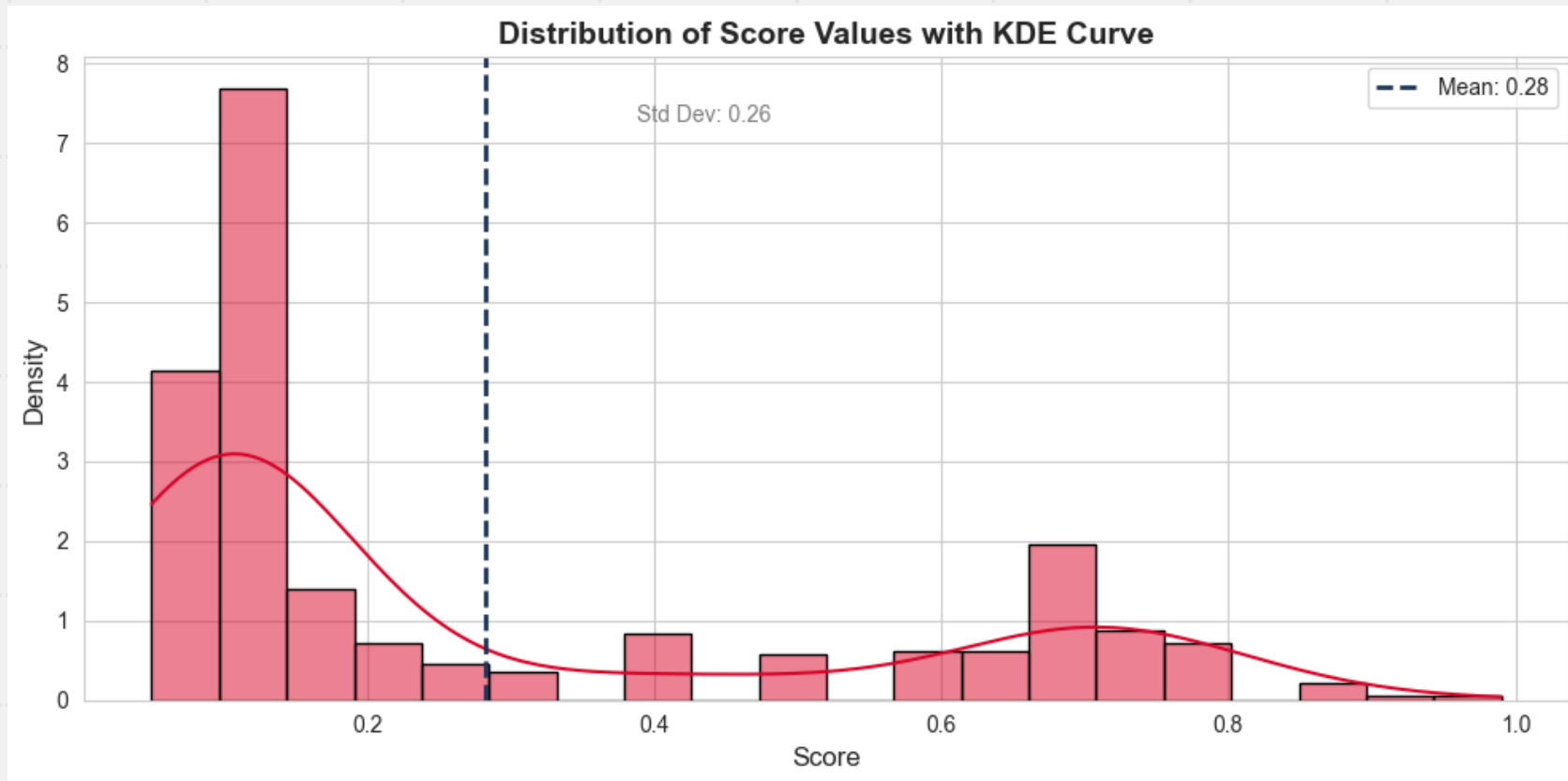
Exploratory Data Analysis



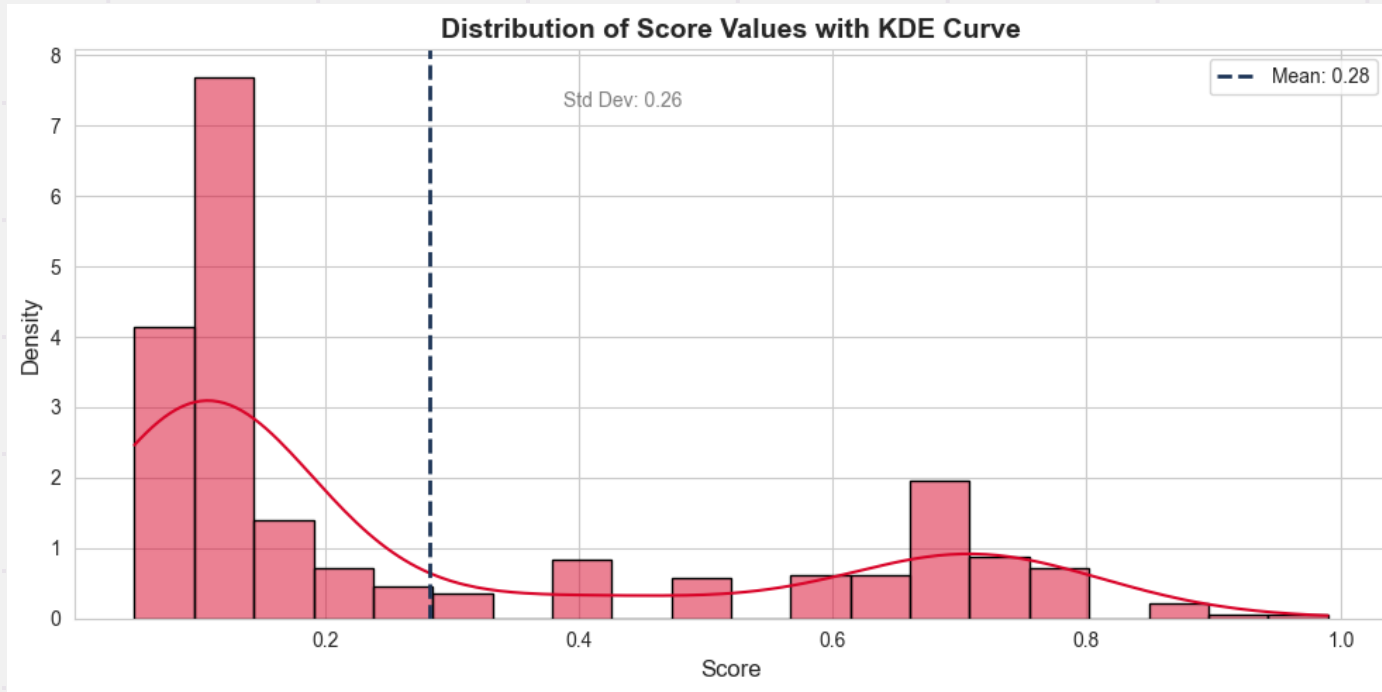
Negative and Neutral have more variety in word counts (likely due to Criticisms, etc)

Positive reviews are much more shorter and stable

Exploratory Data Analysis



Exploratory Data Analysis



Heavily right skewed

- **Negative biased dataset**

- **Average of 0.28 score**

Exploratory Data Analysis

[illegible][illegible]

Exploratory Data Analysis

WordCloud for Label 0

WordCloud for Label 1

WordCloud for Label 2

Label 0 (negative):

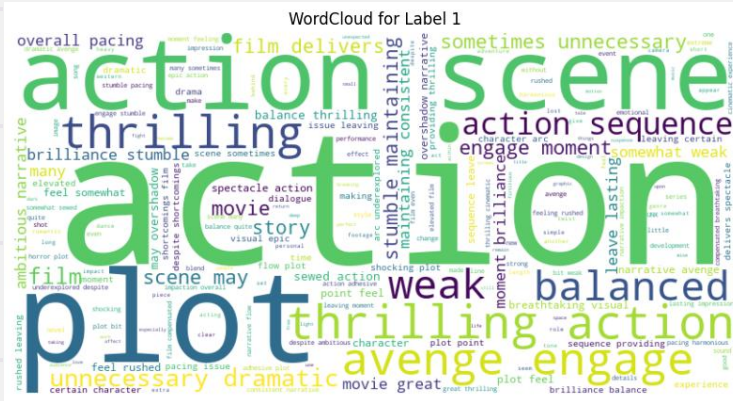
- Has many mixed positive-negative sounding terms (likely to have many mismatched labels)

Label 1 (neutral):

- Contains both positive and negative emotions (normal for neutral)

Label 2 (Positive):

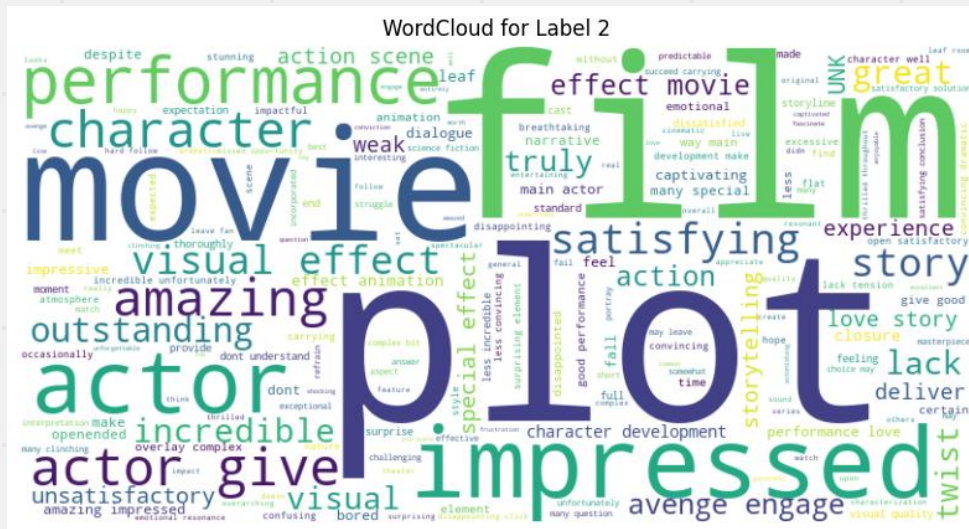
- Many positive emotions with a few minor outliers (likely mislabelled)



- **Has many mixed positive-negative sounding terms**
(likely to have many mismatched labels)

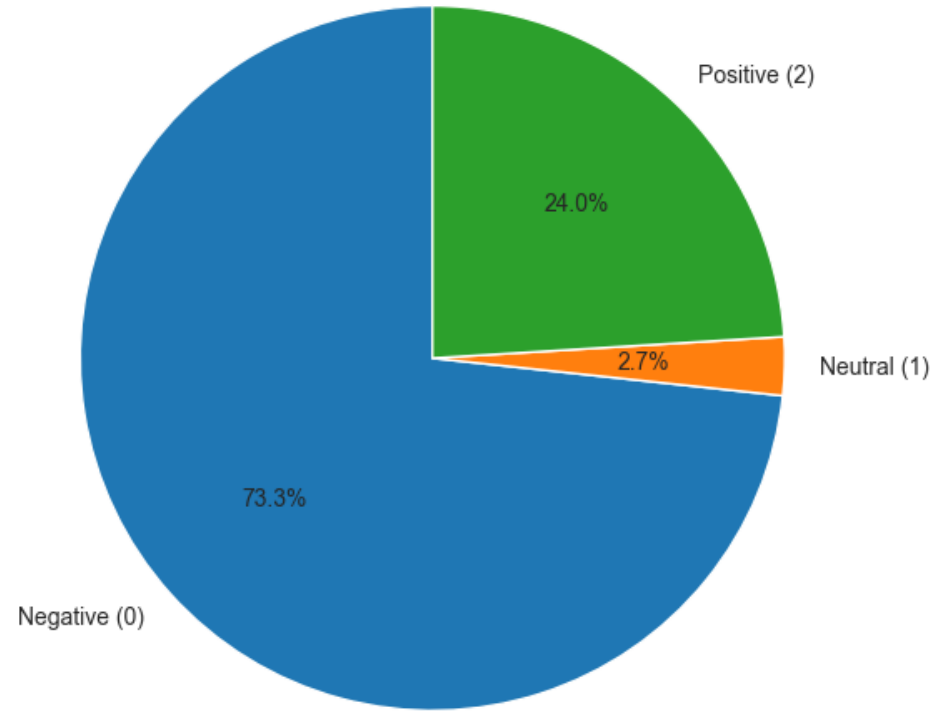
- **Contains both positive and negative emotions (normal for neutral)**

- Many positive emotions with a few minor outliers (likely mislabelled)



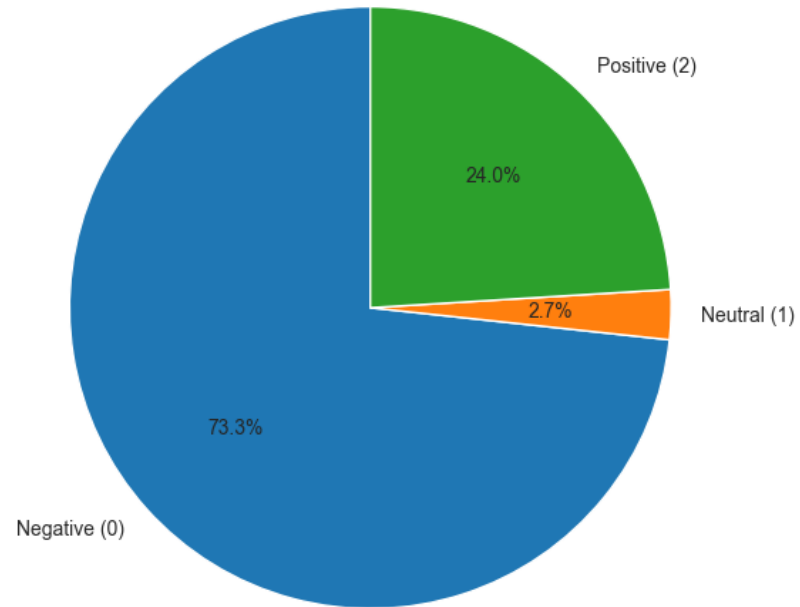
Data Imbalance

Simulated Class Distribution Before Augmentation (Same formula for how score binning will be done)



Data Imbalance

Simulated Class Distribution Before Augmentation (Same formula for how score binning will be done)



- **Extreme class imbalance**
- **Mostly Negative**
- **Many wrong labels.**

Wrong Label fix – Polarity Checker

What is a polarity checker?

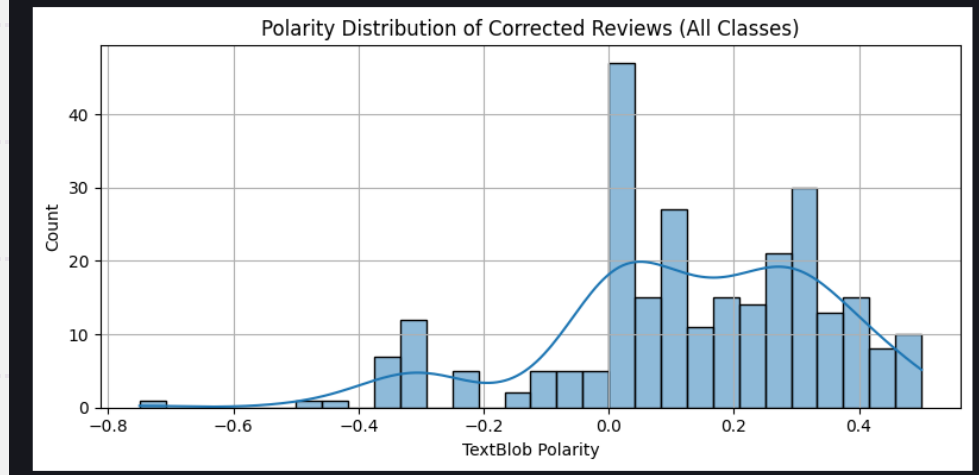
- A tool to determine the tone (sentiment) of a text.
- In Natural Language Processing, it's called sentiment analysis.
- Assigns a score between **-1 & 1** depending on the tone

How I used it:

- I used the polarity checker from the TextBlob Library.
- Checked the tone of the reviews.
- Compared the scores with their respective scores
- Changed **ONLY the extremely different in tones** to preserve as much data integrity as possible as polarities can be wrong.

[INFO] Scores corrected due to polarity mismatch: 97 of 412 (23.54%)
Sample corrected rows:

	Review	Score	polarity
43	storyline quite common turned great action	0.25	0.20
284	visual effect nothing short spectacular truly ...	0.08	0.40
228	film make laugh cry love emotion conveyed	0.15	0.40
299	visual effect leave lasting impact technical b...	0.08	0.25
379	avenge engage chi practical effect work harmon...	0.08	0.14



Wrong Label fix – Polarity Checker

I only corrected reviews where the **text and the score clearly didn't match**.

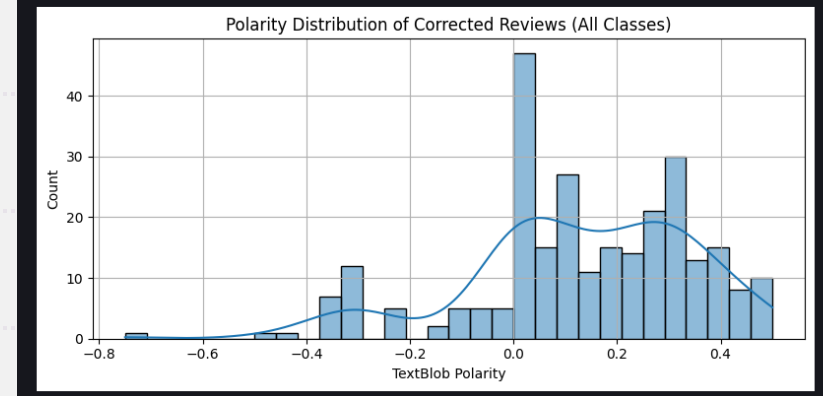
- If the **score was very low** (≤ 0.2) but the **review was very positive** (polarity ≥ 0.5), I changed the score to **high (0.8)**.
- If the **score was very high** (≥ 0.8) but the **review was very negative** (polarity ≤ -0.5), I changed the score to **low (0.2)**.
- If the **score was in the middle** (0.4 to 0.6) and the review was **neutral** (polarity near 0), I slightly adjusted it to **0.5** to make it clearly neutral.

This helped **correct only extreme mismatches** while keeping the dataset reliable.

[INFO] Scores corrected due to polarity mismatch: 97 of 412 (23.54%)

Sample corrected rows:

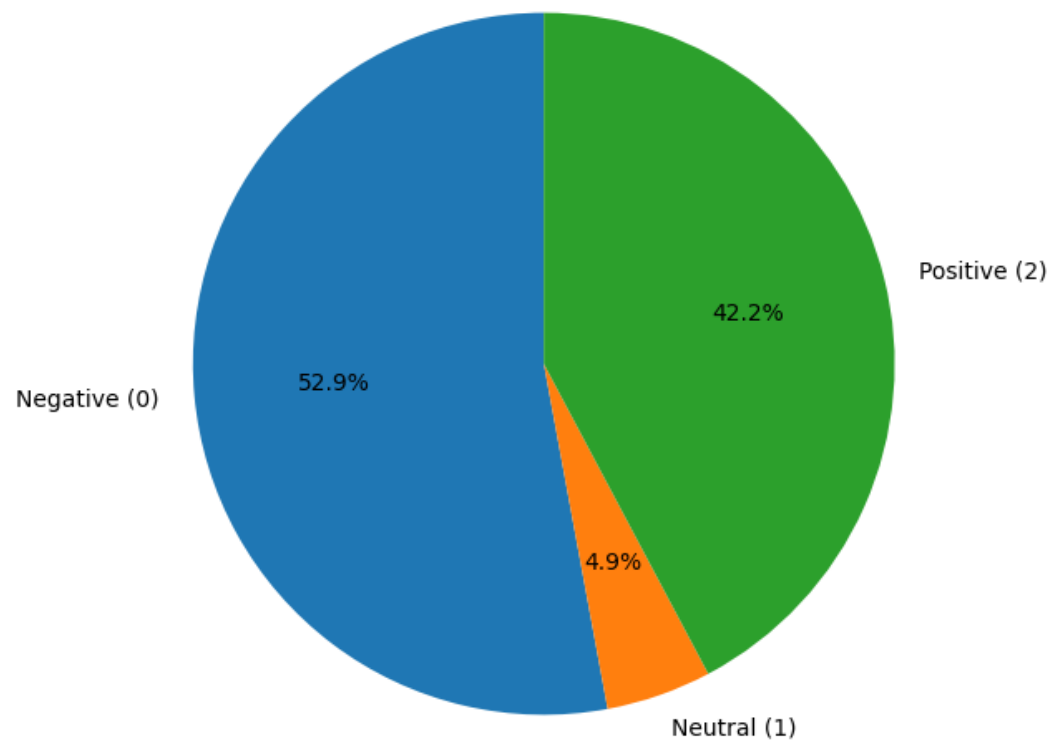
	Review	Score	polarity
43	storyline quite common turned great action	0.25	0.20
284	visual effect nothing short spectacular truly ...	0.08	0.40
228	film make laugh cry love emotion conveyed	0.15	0.40
299	visual effect leave lasting impact technical b...	0.08	0.25
379	avege engage chi practical effect work harmon...	0.08	0.14



Most corrected reviews are neutral, with a few extreme ones (total 23.54% of data were corrected)

Class distribution after label fix

Simulated Class Distribution Before Augmentation (Same formula for how score binning will be done)

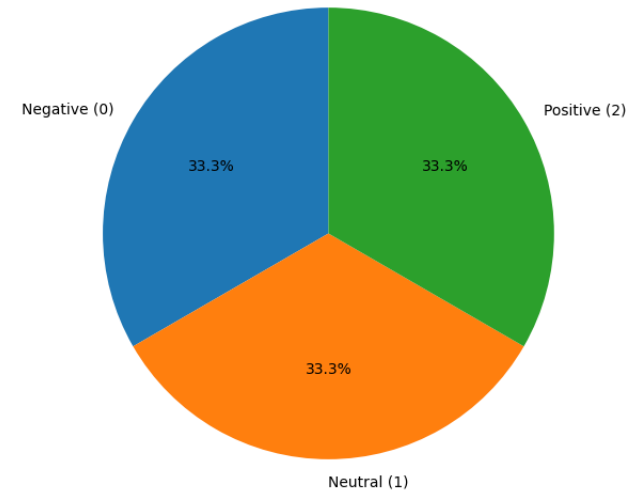


Augmentation for class balance

- Calculated how much data each class has
- Augment them to be equal sized
- Much better than using class weights (shown next slide)

*I used a BERT-based augementer that selects words in each review and replaces them with contextually similar alternatives predicted by BERT. This creates **natural, human-like sentence variations** while preserving meaning.*

Simulated Class Distribution AFTER Augmentation (Same formula for how score binning will be done)



Why not class weights?

Class weights	Augmentation
Only changes class priority	Creates new review texts for the class
Does not give new examples	More language patterns to learn
Passive fix	Makes data richer, not just even

Final Augmentation

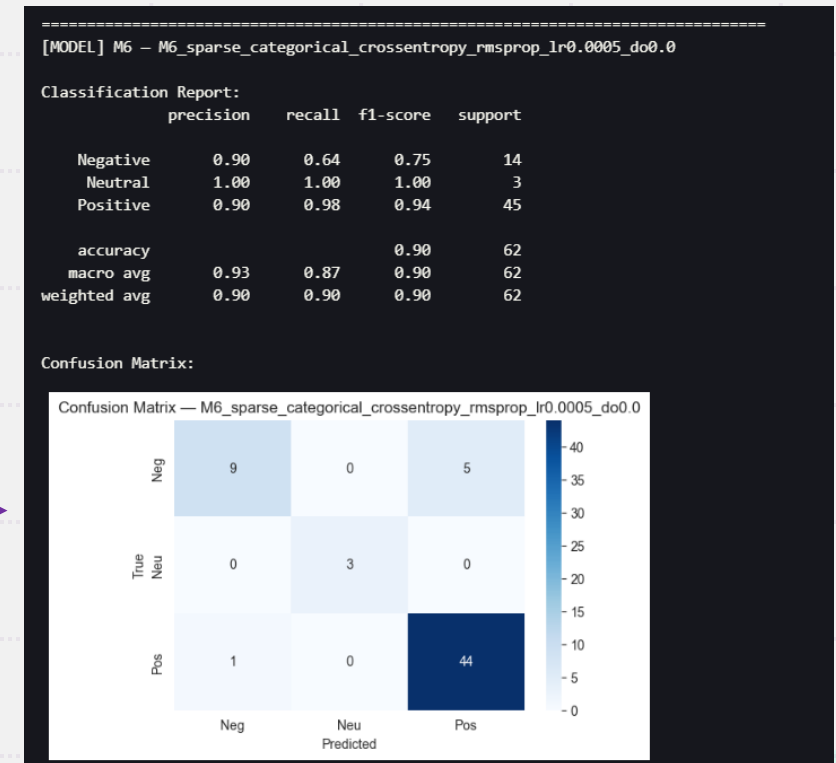
- Augmented the whole dataset again after class balancing
- Class-based augmentation solved class-balancing, but not variation
- Augmenting again introduces new sentence structures and synonyms.

```
[PHASE 2] Starting global stack augmentation...  
[PHASE 2] Global augmentation complete. Final dataset size: (1308, 6)
```

Our final dataset has 1308 rows of data

Why augment whole dataset?

- It's common practice to augment **train data only** to not **risk data leakage**.
- BUT, our dataset is **small**.
- Augmenting only the train dataset creates a **small test and validation size (~40 samples each)**
- Produces **unreliable metrics and confusion matrix** →
- Augmented whole data for the sake of **higher metric reliability** while compensating with **potential slight data leakage (minimal because I'm not duplicating)**



Model Grid Search

- Grid search with the following fixed configs:
 - **50 epochs**
 - **Batch size = 32** (balance between efficiency and stabilization)
 - **ReduceLROnPlateau scheduler** (testing lots of models, helps convergence in all models)
 - **Sparse Categorical Crossentropy** (3 class, expects integer labels)
- Grid search performs search over:
 - **Optimizers (Adam, SGD, Rmsprop, Nadam)**
 - **Learning rates (0.001, 0.0005)**
 - **Dropout rates (0.0, 0.3, 0.5)**

```
TUNING_GRID = {  
    "losses": ["sparse_categorical_crossentropy"],  
    "optimizers": ["adam", "sgd", "rmsprop", "nadam"],  
    "learning_rates": [0.001, 0.0005],  
    "dropout_rates": [0.0, 0.3, 0.5],  
    "epochs": 50  
}
```

Manual Grid-Search vs Keras Tuner

Manual grid search was chosen for its clarity, control, and reproducibility, allowing structured testing of key model configurations under a consistent evaluation pipeline.

- **Manual grid search** - full control of what configuration and what model to test - no hidden search logic
- All combinations and results are **logged (and exported to a csv file)**
- **Keras-Tuner** does all the tuning for you **without showing much reasoning** on why other parameters are not compatible **VS Manual Grid Search** able to show every configuration's **performance**.
- In short, manual grid search offers a **more targeted parameter search** as well as **more insights into what configurations work for this dataset**, while **sacrificing little performance**.

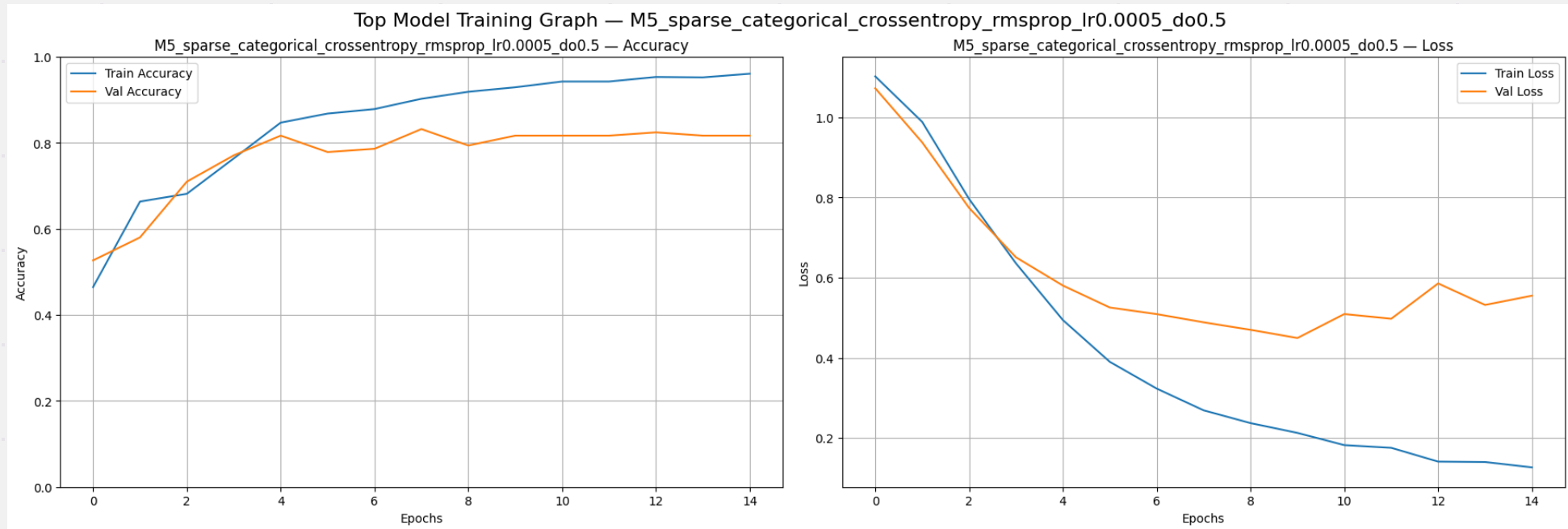
Grid Search – Model list

Model Name	Why I Chose It
M1_SimpleRNN	Baseline Model
M2_LSTM	Captures long-term dependencies
M3_Bidirectional_LSTM	Adds context from both directions
M4_GRU	Faster alternative to LSTM
M5_Bidirectional_GRU	Efficient bidirectional sequence model
M6_Stacked_Bidirectional_LSTM	Deep architecture with bidirectional memory
M7_Stacked_Bidirectional_GRU	Deep GRU-based variant for complex patterns
M8_CNN_LSTM	Captures phrases, and learns their meaning together
M9_Embedding_GlobalAvgPool	Averages the scores of the whole sentence to determine its sentiment

Model results – Top 3

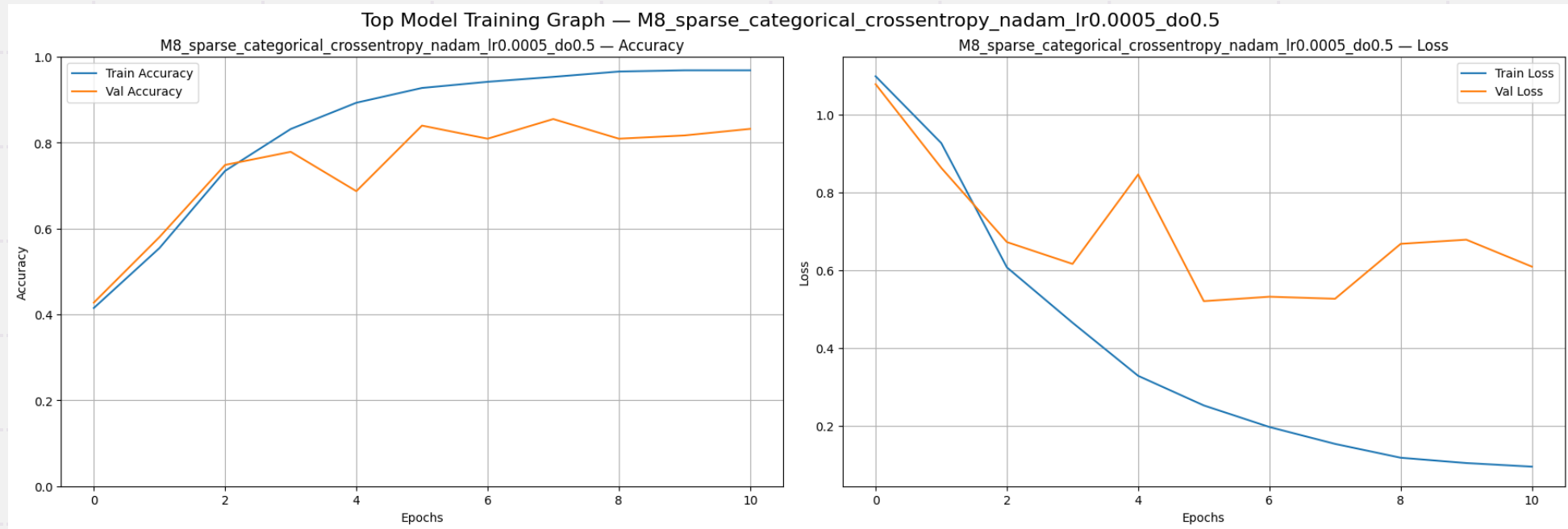
Model Name	Drop-out rate	Learning Rate	Train Accuracy	Validation Accuracy	Test Accuracy	f1_macro
Bidirectional-GRU	0.5	0.0005	94.46%	81.68%	91.60%	91.67%
CNN-LSTM	0.5	0.0005	94.26%	83.97%	90.84%	90.86%
Stacked Bidirectional-GRU	0.5	0.0005	95.98%	83.97%	90.84%	90.83%

Model results – Training history Plot (top 3)



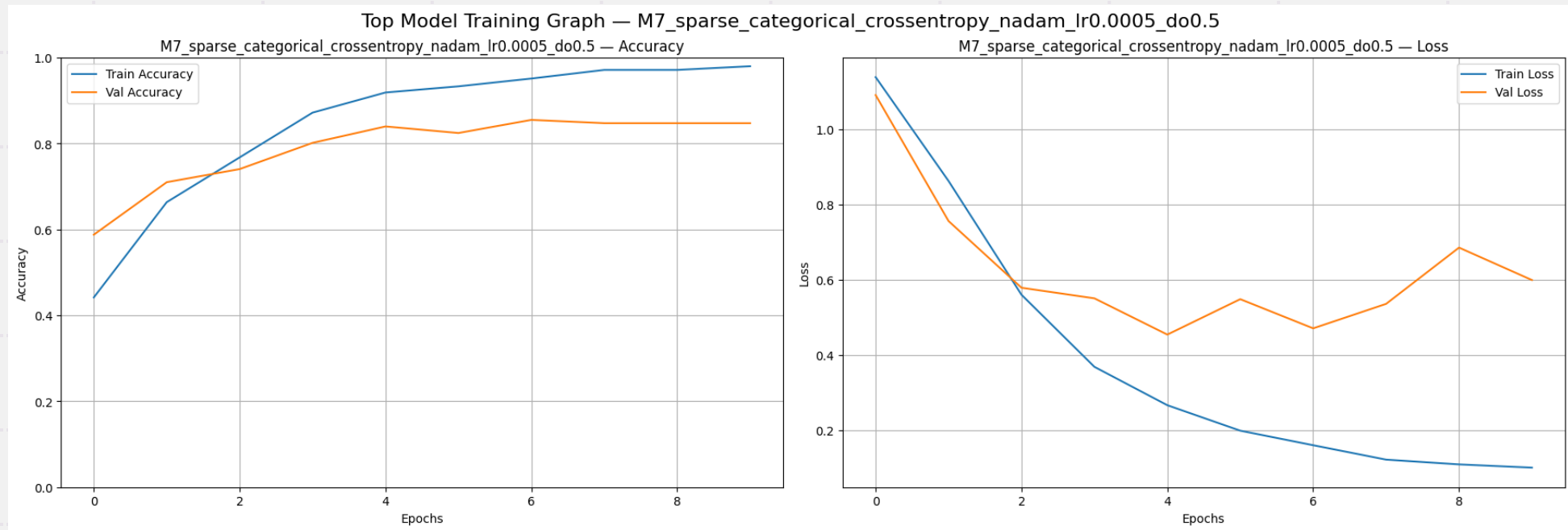
Mild Overfitting
Dropout 0.5 helped delay
Could try increasing regularization

Model results – Training history Plot (top 3)



Excellent on learning train data
Overfitting begins on epoch 4-5
Validation still remains high.

Model results – Training history Plot (top 3)



Same as before
Overfit starts a bit earlier
Solid model, but flatlines early.

Model improvement phase 2: increase regularization

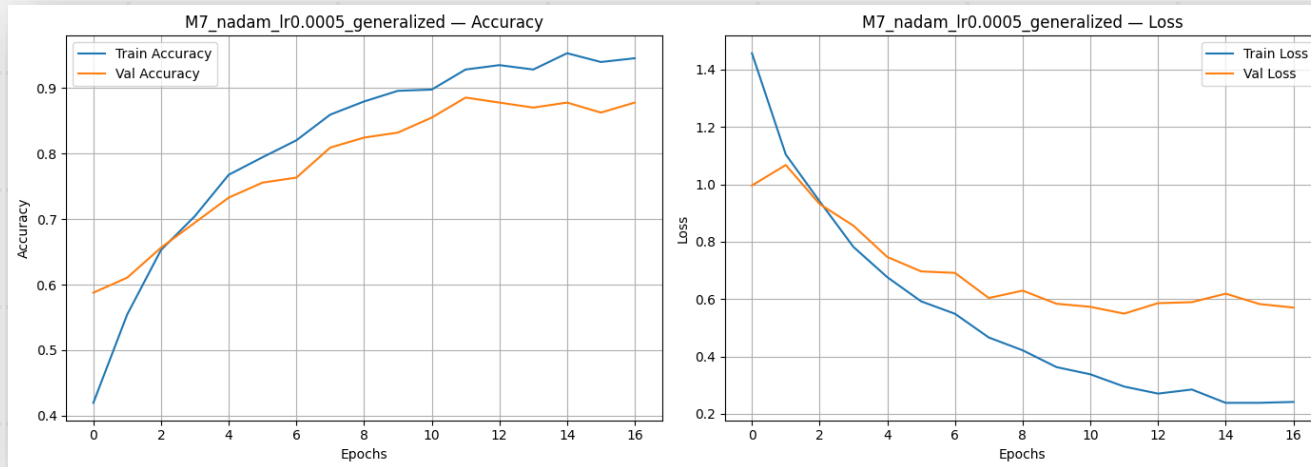
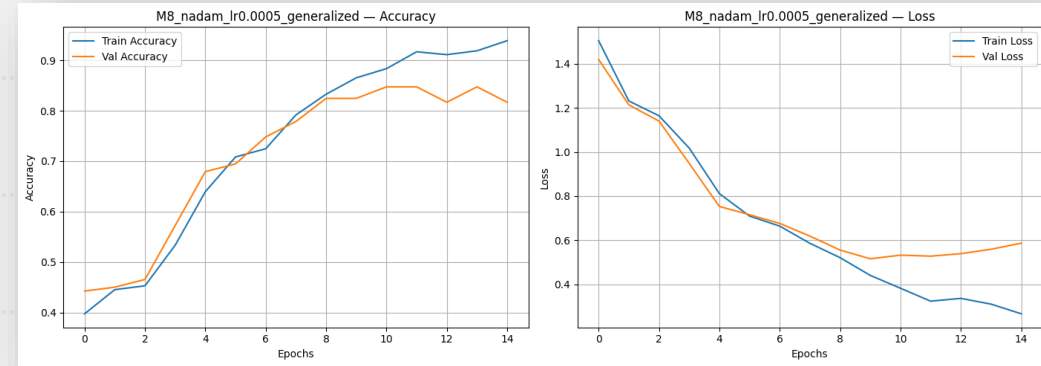
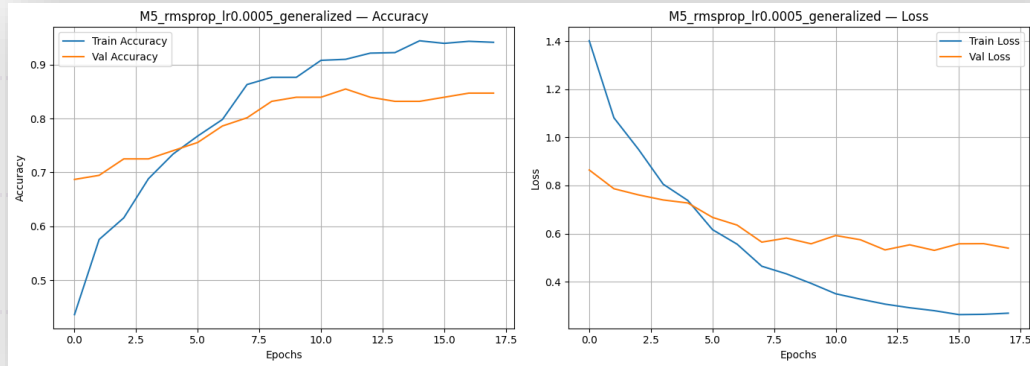
What I added:

Component	Effect
L2 Embedding	Limits embedding weight growth
GaussianNoise	Improves robustness to input noise
LayerNormalization	Stabilizes activations (mean ~ 0 , S.D ~ 1)
L1_L2 Regularizer	Combats overfitting and adds sparsity
Dense(64, relu)	Adds nonlinearity with regularized capacity
BatchNormalization	Normalizes and accelerates training
GlobalAvgPooling (M9)	Reduces complexity of sequence input

Model improvement phase 2: increase regularization

Top 3 Models — Retrained with Generalized Architecture (Fixed Dropout, No Label Smoothing)						
	experiment_id	train_accuracy	val_accuracy	test_accuracy	f1_macro	mean_accuracy
0	M5_rmsprop_lr0.0005_generalized	96.75%	83.97%	85.50%	85.25%	88.74%
1	M8_nadam_lr0.0005_generalized	93.98%	82.44%	88.55%	88.62%	88.32%
2	M7_nadam_lr0.0005_generalized	96.75%	88.55%	82.44%	81.87%	89.25%

Model improvement phase 2: increase regularization



✓ Regularization did improve

But what about our accuracies?

Model improvement phase 2: increase regularization

F1 Score & Mean Accuracy — Original vs Generalized

	model_id	f1_macro	f1_macro_generalized	f1_diff	mean_accuracy	mean_accuracy_generalized	mean_accuracy_diff
0	M5	91.67%	85.25%	-6.42%	89.25%	88.74%	-0.51%
1	M8	90.86%	88.62%	-2.24%	89.69%	88.32%	-1.37%
2	M7	90.83%	81.87%	-8.96%	90.26%	89.25%	-1.01%

Train Accuracy — Original vs Generalized

	model_id	train_accuracy	train_accuracy_generalized	train_acc_diff
0	M5	94.46%	96.75%	2.29%
1	M8	94.26%	93.98%	-0.28%
2	M7	95.98%	96.75%	0.77%

Validation & Test Accuracy — Original vs Generalized

	model_id	val_accuracy	val_accuracy_generalized	val_acc_diff	test_accuracy	test_accuracy_generalized	test_acc_diff
0	M5	81.68%	83.97%	2.29%	91.60%	85.50%	-6.10%
1	M8	83.97%	82.44%	-1.53%	90.84%	88.55%	-2.29%
2	M7	83.97%	88.55%	4.58%	90.84%	82.44%	-8.40%

Model improvement phase 2: increase regularization

F1 Score & Mean Accuracy — Original vs Generalized							
model_id	f1_macro	f1_macro_generalized	f1_diff	mean_accuracy	mean_accuracy_generalized	mean_accuracy_diff	
0	M5	91.67%	85.25%	-6.42%	89.25%	88.74%	-0.51%
1	M8	90.86%	88.62%	-2.24%	89.69%	88.32%	-1.37%
2	M7	90.83%	81.87%	-8.96%	90.26%	89.25%	-1.01%

Train Accuracy — Original vs Generalized				
model_id	train_accuracy	train_accuracy_generalized	train_acc_diff	
0	M5	94.46%	96.75%	2.29%
1	M8	94.26%	93.98%	-0.28%
2	M7	95.98%	96.75%	0.77%

Validation & Test Accuracy — Original vs Generalized							
model_id	val_accuracy	val_accuracy_generalized	val_acc_diff	test_accuracy	test_accuracy_generalized	test_acc_diff	
0	M5	81.68%	83.97%	2.29%	91.60%	85.50%	-6.10%
1	M8	83.97%	82.44%	-1.53%	90.84%	88.55%	-2.29%
2	M7	83.97%	88.55%	4.58%	90.84%	82.44%	-8.40%

Even though our regularization improved.

It compensated with lower accuracies and F1 score

Hence, we'll use back our original models

Reliability test of our best model – Top model (Tested top 3)

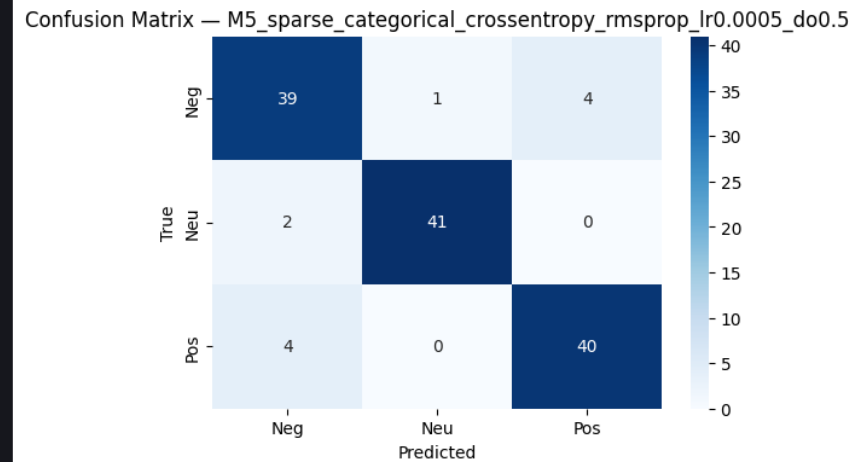
- **Highly accurate and balanced, especially for Neutral and Positive classes.**
- **Slightly weaker in Negative class precision**
- **Minimal confusion and very clean prediction boundaries shown in the matrix.**
- × **Not entirely reliable due to potential minimal data leakage from augmentation, but useful – requires further testing**

[MODEL] M5 – M5_sparse_categorical_crossentropy_rmsprop_lr0.0005_do0.5

Classification Report:

	precision	recall	f1-score	support
Negative	0.87	0.89	0.88	44
Neutral	0.98	0.95	0.96	43
Positive	0.91	0.91	0.91	44
accuracy			0.92	131
macro avg	0.92	0.92	0.92	131
weighted avg	0.92	0.92	0.92	131

Confusion Matrix:



Real world reliability test - Testing on real world data

```
1/1 [=====] - 0s 30ms/step
[INFO] Using top model: M5_sparse_categorical_crossentropy_rmsprop_lr0.0005_do0.5
-----
Review      : Not bad, but not great either.
Prediction  : Neutral (0.76 confidence)
-----
Review      : I wouldn't watch it again.
Prediction  : Positive (0.87 confidence)
-----
Review      : This was fine. Some moments were dull.
Prediction  : Neutral (0.37 confidence)
-----
Review      : Really disappointing after the hype.
Prediction  : Negative (0.59 confidence)
-----
Review      : Honestly? I kind of liked it.
Prediction  : Positive (0.75 confidence)
```

Real world reliability test - Testing on real world data

```
1/1 [=====] - 0s 30ms/step  
[INFO] Using top model: M5_sparse_categorical_crossentropy_rmsprop_lr0.0005_do0.5
```

✓
Review : Not bad, but not great either.
Prediction : Neutral (0.76 confidence)

✗
Review : I wouldn't watch it again.
Prediction : Positive (0.87 confidence)

✓
Review : This was fine. Some moments were dull.
Prediction : Neutral (0.37 confidence)

✓
Review : Really disappointing after the hype.
Prediction : Negative (0.59 confidence)

✓
Review : Honestly? I kind of liked it.
Prediction : Positive (0.75 confidence)

4/5 Correct

- Low confidence but correct
- Overly optimistic on 2nd review
 - Could be due to labelling issues
- Great performance considering our restricted dataset

Improvement methods

As seen, our dataset is INCREDIBLY small.

- Most of our data is augmented (synthetic)
- Labels are very messy

Model Improvements will happen if:

- Data size increases
- Much more correct and reliable labelling (manually verified labellings)
- More variety of data (with sarcasm, less-obvious reviews for models to capture)
- More data for the small, underrepresented classes



End of presentation